



CICLO FORMATIVO DE GRADO SUPERIOR
DESARROLLO DE APLICACIONES
MULTIPLATAFORMA

Proyecto de Fin de Ciclo

**Diseño e implementación de una plataforma web para el
análisis y recomendación de películas mediante aprendizaje
automático y embeddings semánticos**

Daniel Esteban Moreno

Tutor: Arantxa Herrera

Curso: 2025/2026

Resumen

El presente TFG documenta el diseño, la implementación y la validación de una plataforma web para la exploración y recomendación de películas. El sistema parte de un dataset público de más de cuarenta y cinco mil películas, sobre el que se aplica un pipeline automatizado de limpieza, transformación y carga en una base de datos PostgreSQL con extensión pgvector. El componente central del proyecto es un sistema de recomendación basado en contenido que utiliza embeddings semánticos de 768 dimensiones generados por el modelo preentrenado all-mpnet-base-v2 (arquitectura Transformer) a partir de las sinopsis de las películas, indexados mediante HNSW para búsquedas de similitud en tiempo sublineal. El backend, desarrollado en Kotlin con el framework Ktor, expone cinco endpoints HTTP que cubren listado paginado con filtros, detalle, búsqueda textual, recomendaciones por similitud semántica y listado de géneros, con una suite de treinta pruebas de integración. La interfaz web, desarrollada en Kotlin con Compose Multiplatform compilado a JavaScript, implementa listado con scroll infinito, detalle de película con recomendaciones, búsqueda con autocompletado en tiempo real y alternancia de tema claro/oscuro. El módulo compartido entre backend y frontend, habilitado por Kotlin Multiplatform, elimina la duplicación de modelos de datos entre servidor y cliente. El sistema se sirve como un proceso único desde Ktor, facilitando su despliegue y demostración.

Abstract

This final degree project documents the design, implementation and validation of a web platform for movie exploration and recommendation. The system processes a public dataset of over forty-five thousand movies through an automated cleaning, transformation and loading pipeline into a PostgreSQL database with the pgvector extension. The core component is a content-based recommendation system using 768-dimensional semantic embeddings generated by the pre-trained all-mpnet-base-v2 model (Transformer architecture) from movie synopses, indexed with HNSW for sublinear similarity search. The backend, developed in Kotlin with the Ktor framework, exposes five HTTP endpoints covering paginated listing with filters, detail, text search, semantic similarity recommendations and genre listing, backed by thirty integration tests. The web interface,

developed in Kotlin with Compose Multiplatform compiled to JavaScript, implements infinite-scroll listing, movie detail with recommendations, real-time autocomplete search and dark/light theme toggle. A shared module between backend and frontend, enabled by Kotlin Multiplatform, eliminates data model duplication between server and client. The system is served as a single process from Ktor, simplifying deployment and demonstration.

Agradecimientos

Quiero expresar mi agradecimiento a todas las personas que han contribuido, de una forma u otra, a que este proyecto llegue a buen puerto.

En primer lugar, a mi tutora Arantxa Herrera, por su orientación, su disponibilidad y sus consejos a lo largo de todo el desarrollo del trabajo.

A los profesores del ciclo, por los conocimientos transmitidos durante este curso que han hecho posible este proyecto.

A Jorge Muñoz, mi tutor de prácticas, por la oportunidad de realizar las prácticas curriculares en un entorno profesional real y por permitirme familiarizarme con las tecnologías que han sido la base de este trabajo.

Por último, y de manera especial, a mi familia, por su paciencia, su confianza y su apoyo incondicional en cada etapa de mi formación.

A todos, gracias.

4. Justificación del proyecto.....	5
5.1. Contexto del dominio: el análisis de datos cinematográficos.....	6
5.2. Fundamentos teóricos de los sistemas de recomendación.....	7
5.3. Embeddings semánticos y búsqueda por similitud vectorial.....	8
6. Objetivos.....	10
Objetivo general.....	10
Objetivos específicos.....	10
7. Desarrollo.....	11
Tecnologías empleadas.....	11
Pipeline de datos: Python 3.12.....	11
Sistema gestor de base de datos: PostgreSQL 16 con pgvector.....	11
Orquestación de contenedores: Podman.....	12
Backend: Kotlin con framework Ktor.....	12
Frontend: Kotlin Compose Multiplatform compilado a JavaScript.....	12
Modelo de embeddings: sentence-transformers.....	12
Herramientas auxiliares.....	13
7.1 Planificar.....	13
7.1.1 Metodología de trabajo adoptada.....	13
7.1.2 Planificación temporal.....	14
7.1.3 Estrategia de priorización por riesgo.....	14
7.2 Buscar.....	15
7.2.1 Búsqueda y selección del dataset.....	15
7.2.2 Estructura del dataset seleccionado.....	16
7.2.3 Metodología de inspección.....	16
7.2.4 Resultados de la inspección por archivo.....	17
7.2.5 Catálogo de problemas identificados.....	18
7.2.6 Decisiones de diseño consolidadas.....	19
7.3 Ejecutar.....	20
7.3.1 Limpieza y transformación de datos.....	20
7.3.2 Diseño de la base de datos y carga de datos.....	22
7.3.3 Desarrollo del backend.....	26
7.3.4 Desarrollo del sistema de recomendación.....	29
7.3.5 Desarrollo de la aplicación cliente.....	31
8. Conclusiones.....	35
8.1 Grado de cumplimiento de los objetivos.....	35
8.2 Lecciones aprendidas.....	36
8.3 Ampliaciones futuras.....	37

4. Justificación del proyecto

El presente TFG nace de la confluencia de tres factores: la experiencia profesional adquirida durante el periodo de prácticas curriculares en empresa, el interés personal hacia las tecnologías emergentes basadas en inteligencia artificial, y la voluntad de consolidar de forma integrada los conocimientos adquiridos a lo largo del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma.

Durante el periodo de formación en centros de trabajo, el alumno ha participado en el desarrollo de un sistema de análisis y visualización de datos sobre empresas tecnológicas de la aceleradora Y Combinator, lo que le ha permitido familiarizarse con un stack tecnológico moderno basado en Kotlin para el lenguaje principal, Ktor para el desarrollo backend, Compose Multiplatform para el frontend web, y PostgreSQL como sistema gestor de bases de datos relacional. Este conjunto de competencias se ha aplicado, sin embargo, a un dominio muy específico y dentro del marco operativo de la empresa, sin que el alumno haya tenido la oportunidad de abordar el ciclo de desarrollo completo de forma autónoma ni de explorar técnicas diferentes asociadas a la inteligencia artificial moderna.

El presente proyecto persigue trasladar y consolidar ese conjunto de competencias técnicas a un dominio diferente —el análisis y la recomendación de productos cinematográficos— incorporando además una capa adicional de complejidad tecnológica mediante la integración de modelos de lenguaje y técnicas de búsqueda vectorial basadas en embeddings semánticos. La elección del dominio cinematográfico responde a la disponibilidad de un dataset público de calidad contrastada (**The Movies Dataset** publicado en **Kaggle** bajo licencia CC0), a su atractivo visual para una aplicación interactiva, y a la riqueza de información estructurada que permite plantear simultáneamente tareas de exploración, predicción y recomendación.

La integración de embeddings semánticos resulta especialmente relevante dado el creciente protagonismo de estas tecnologías en el panorama actual del desarrollo de software, donde la búsqueda por similitud vectorial se ha convertido en un componente habitual de sistemas de recomendación, asistentes conversacionales y motores de búsqueda inteligente. Su incorporación al proyecto constituye además una preparación natural para los estudios de máster especializados en inteligencia artificial que el alumno tiene previsto cursar tras la finalización del ciclo formativo.

En definitiva, el TFG se concibe como un puente entre el conocimiento adquirido durante el ciclo, la experiencia práctica desarrollada en la empresa de prácticas, y la formación especializada futura, materializado en una aplicación web completa de extremo a extremo que demuestra el dominio integrado de las competencias propias del título.

5. Introducción

5.1. Contexto del dominio: el análisis de datos cinematográficos

La industria cinematográfica produce, cataloga y consume una cantidad creciente de información estructurada sobre las películas y los profesionales que las realizan. Plataformas como The Movie Database (TMDb), Internet Movie Database (IMDb), Letterboxd o JustWatch han desarrollado bases de datos con decenas de millones de registros que abarcan títulos, sinopsis, equipos artísticos y técnicos, presupuestos, recaudaciones, valoraciones y otros metadatos asociados a cada producción. Esta información, históricamente accesible únicamente a profesionales del sector, ha pasado en las últimas décadas a estar disponible al público general en forma de aplicaciones web interactivas, lo que ha generado una nueva categoría de productos digitales orientados al descubrimiento, la consulta y la recomendación de contenido audiovisual.

Las soluciones existentes en este ámbito se pueden clasificar de forma simplificada en cuatro categorías. En primer lugar, los catálogos descriptivos como IMDb o TMDb, cuya función principal es servir como fuente exhaustiva de información de referencia sobre las producciones. En segundo lugar, las plataformas sociales como Letterboxd, que añaden una capa de interacción comunitaria donde los usuarios valoran, comentan y elaboran listas personales. En tercer lugar, los servicios agregadores como JustWatch, cuya propuesta de valor es informar al usuario de en qué plataforma de streaming puede consumir cada título. Finalmente, las propias plataformas de streaming (Netflix, Prime Video, Filmin) incorporan sistemas internos de recomendación cuya finalidad principal es maximizar el tiempo de visionado de cada usuario.

Existe un denominador común a todas las soluciones anteriores: todas ellas combinan, en mayor o menor grado, funcionalidades de catálogo (búsqueda, filtrado, ficha de detalle) con funcionalidades de recomendación (sugerencias personalizadas o no, contenido relacionado, listados temáticos). El presente proyecto se inscribe dentro de esta misma categoría general, si bien con tres diferencias significativas respecto a las soluciones comerciales referidas. Primera, su carácter exclusivamente académico, lo que permite priorizar la transparencia de las técnicas empleadas frente al valor comercial de las

mismas. Segunda, su enfoque sobre un dataset abierto y reproducible, lo que facilita la verificación de los resultados por parte de terceros. Tercera, la integración explícita de técnicas modernas de procesamiento del lenguaje natural (concretamente, embeddings semánticos) como mecanismo central del sistema de recomendación, en lugar de los enfoques tradicionales basados en metadatos categóricos o en filtrado colaborativo puro.

5.2. Fundamentos teóricos de los sistemas de recomendación

Un sistema de recomendación es, en términos generales, un programa informático capaz de sugerir a un usuario elementos de un catálogo que probablemente le resulten relevantes, partiendo de información previa sobre el usuario, sobre los propios elementos, o sobre ambos. La literatura especializada distingue tradicionalmente tres familias principales de sistemas de recomendación, con caracterizaciones bien establecidas desde los trabajos seminales de la década de 1990.

El filtrado colaborativo parte de la hipótesis de que usuarios con preferencias similares en el pasado tenderán a coincidir en sus preferencias futuras. Para emitir una recomendación, el sistema identifica usuarios cuyas valoraciones previas se asemejan a las del usuario activo y agrega las valoraciones de aquellos. Esta familia incluye tanto enfoques basados en vecindad (memory-based) como enfoques basados en modelos como la descomposición en valores singulares (SVD) o sus extensiones. Sus principales fortalezas son la capacidad de descubrir relaciones no evidentes a partir del propio comportamiento agregado de los usuarios, sin necesidad de modelar explícitamente las características de los elementos. Sus principales limitaciones son el llamado **cold start**, esto es, la incapacidad para emitir recomendaciones cuando un usuario o un elemento dispone de pocas o ninguna valoración previa, así como la dependencia de un volumen significativo de datos de interacción.

La recomendación basada en contenido opera de forma radicalmente distinta. En lugar de analizar el comportamiento agregado de los usuarios, este enfoque construye una representación vectorial de cada elemento del catálogo a partir de sus atributos intrínsecos (en el caso de las películas: géneros, sinopsis, palabras clave, equipo creativo, etcétera), y emite recomendaciones devolviendo los elementos cuya representación es más cercana, según alguna métrica de distancia, a la del elemento de referencia. Esta familia no presenta el problema del **cold start** en el lado de los elementos, dado que un elemento puede ser representado y recomendado desde el momento mismo de su incorporación al catálogo, antes de recibir valoración alguna. Sus limitaciones se

relacionan con la calidad de la representación vectorial empleada: si dicha representación captura únicamente atributos superficiales (por ejemplo, una bolsa de palabras sobre la sinopsis), las recomendaciones tienden a ser superficiales también.

Los sistemas híbridos combinan ambos enfoques de formas diversas, generalmente para mitigar las limitaciones de cada uno por separado. La literatura reciente reporta consistentemente que los sistemas híbridos producen mejores resultados que los enfoques puros, si bien a costa de mayor complejidad de implementación y mayor coste computacional.

El presente proyecto se centra en un sistema de recomendación basado en contenido, con la particularidad de emplear una representación vectorial moderna basada en embeddings semánticos. Esta decisión se justifica por dos motivos. Primero, la ausencia de un sistema de cuentas de usuario en el alcance del proyecto hace inviable un enfoque colaborativo. Segundo, los embeddings semánticos modernos producen representaciones vectoriales sustancialmente más ricas que las técnicas tradicionales basadas en bolsa de palabras o TF-IDF, lo que permite obtener recomendaciones de calidad notable a partir únicamente del contenido textual disponible.

5.3. Embeddings semánticos y búsqueda por similitud vectorial

Un embedding es una representación numérica densa y de dimensión fija de un objeto cualquiera —en el contexto de este proyecto, de un texto— que tiene la propiedad de preservar relaciones semánticas en el espacio vectorial resultante. Dicho de forma intuitiva: dos textos con significado parecido producen vectores cercanos en dicho espacio, y dos textos con significado distinto producen vectores alejados. Esta propiedad permite resolver problemas de búsqueda semántica y de recomendación por similitud que serían difíciles o imposibles de abordar con técnicas léxicas tradicionales.

La idea de representar palabras como vectores en un espacio continuo se consolidó con la publicación de Word2Vec (Mikolov et al., 2013), que demostró que entrenando una red neuronal poco profunda para predecir palabras a partir de su contexto era posible obtener representaciones vectoriales que capturaban relaciones semánticas y sintácticas sorprendentemente ricas. Esta línea evolucionó posteriormente hacia modelos contextuales como ELMo y, sobre todo, hacia los modelos basados en la arquitectura Transformer (Vaswani et al., 2017) y sus desarrollos posteriores como BERT (Devlin et al., 2018). El paso siguiente, particularmente relevante para sistemas de recomendación a nivel de documento, lo constituye Sentence-BERT (Reimers y Gurevych, 2019), una

adaptación de BERT diseñada específicamente para producir embeddings de oraciones o párrafos completos optimizados para tareas de similitud.

La métrica habitual para medir la cercanía entre dos embeddings es la similitud coseno, que se define como el coseno del ángulo formado por los dos vectores en el espacio. Sus valores oscilan entre -1 y $+1$, donde $+1$ indica vectores idénticos en dirección, 0 indica vectores ortogonales y -1 indica vectores opuestos. La similitud coseno presenta dos ventajas prácticas frente a otras métricas como la distancia euclídea: por un lado, es invariante a la magnitud del vector, lo que evita que textos de longitud distinta produzcan distancias artificialmente grandes; por otro, es computacionalmente eficiente, especialmente cuando los vectores se almacenan ya normalizados.

Para que el sistema de recomendación funcione en tiempo razonable cuando el catálogo crece, no basta con almacenar los embeddings y calcular la similitud frente a cada elemento del catálogo de forma exhaustiva. Es necesario emplear estructuras de datos especializadas que permitan resolver consultas de tipo **k-nearest neighbors** (k-NN) en tiempo sublineal. En el contexto del presente proyecto, esta funcionalidad se aporta mediante la extensión **pgvector** para PostgreSQL, que añade al sistema gestor de bases de datos un tipo de datos 'vector' junto con operadores de distancia e índices especializados.

Dos familias principales de índices están disponibles en pgvector para acelerar las búsquedas de vecindad. La primera, **IVFFlat** (Inverted File with Flat compression), particiona el espacio vectorial en regiones mediante clustering y consulta únicamente las regiones más prometedoras; requiere especificar el número de particiones como parámetro de construcción. La segunda, **HNSW** (Hierarchical Navigable Small World), construye un grafo navegable en múltiples capas inspirado en los trabajos de Malkov y Yashunin (2018), donde la búsqueda procede descendiendo desde una capa superior dispersa hasta una capa inferior densa, alcanzando el vecino objetivo en tiempo logarítmico. HNSW ofrece habitualmente mejor recall que IVFFlat a costa de un mayor tiempo de construcción y un mayor consumo de memoria; para el orden de magnitud del catálogo del presente proyecto (decenas de miles de vectores), ambos parámetros resultan asumibles, y HNSW es la opción seleccionada.

El uso de pgvector permite mantener los embeddings dentro de la misma base de datos relacional que contiene el resto de la información del sistema, evitando la introducción de una base de datos adicional especializada en vectores y simplificando significativamente la arquitectura del sistema.

6. Objetivos

Objetivo general

Desarrollar una plataforma web para el análisis exploratorio y la recomendación personalizada de películas, aplicando técnicas de aprendizaje automático y embeddings semánticos sobre un dataset público de información cinematográfica.

Objetivos específicos

- **Diseñar e implementar un pipeline de limpieza y transformación de datos** que resuelva de forma documentada los problemas de calidad presentes en el dataset original, incluyendo la normalización de campos anidados en formato JSON, el tratamiento de valores ausentes y la corrección de inconsistencias temporales.
- **Modelar una base de datos relacional** que represente adecuadamente las entidades del dominio (películas, personas, géneros, compañías productoras) junto con sus relaciones, aplicando los principios de normalización y diseñando los índices necesarios para garantizar un rendimiento adecuado de las consultas.
- **Desarrollar un servidor backend** con arquitectura por capas (rutas, repositorios, modelos) que exponga los datos del sistema mediante una interfaz REST documentada, garantizando separación de responsabilidades y facilidad de mantenimiento.
- **Implementar una aplicación web cliente** que ofrezca al usuario al menos cuatro vistas funcionales (listado con filtros, ficha de detalle de película, panel analítico, vista de recomendaciones) con una experiencia de usuario coherente y un diseño visual cuidado.
- **Implementar un sistema de recomendación basado en embeddings semánticos** extraídos a partir de las sinopsis de las películas, almacenando dichos embeddings en una base de datos vectorial y devolviendo recomendaciones por similitud coseno en tiempo razonable.
- **Integrar los modelos desarrollados dentro de la aplicación web** de forma transparente al usuario final, ofreciendo las recomendaciones de películas similares dentro del flujo de navegación natural.

- **Documentar de forma completa el proyecto** mediante una memoria técnica que recoja las decisiones de diseño adoptadas, los problemas encontrados durante el desarrollo y las soluciones aplicadas a los mismos.

7. Desarrollo

Tecnologías empleadas

La selección del conjunto de tecnologías empleadas en el proyecto responde a tres criterios combinados: la idoneidad técnica de cada herramienta para el problema concreto que debe resolver, la continuidad respecto a las competencias adquiridas durante el periodo de prácticas curriculares (lo que reduce la curva de aprendizaje y permite concentrar el esfuerzo en los aspectos verdaderamente novedosos del proyecto), y la vigencia de cada herramienta en el ecosistema profesional actual, garantizando que las competencias aplicadas tengan valor más allá del contexto académico.

Pipeline de datos: Python 3.12

El pipeline de limpieza, transformación y análisis del dataset se implementa en Python, versión 3.12. La elección de Python para esta capa del proyecto responde a la práctica establecida en el sector: el ecosistema de análisis de datos en Python (pandas, NumPy, scikit-learn, matplotlib, sentence-transformers) carece de equivalente directo en otros lenguajes en términos de madurez y de comunidad.

La gestión del entorno virtual y de las dependencias se realiza mediante uv, una herramienta moderna desarrollada por Astral que reemplaza a las combinaciones tradicionales de ``pip`` + ``venv`` + ``pip-tools``. Su elección se justifica por su velocidad notablemente superior, su garantía de reproducibilidad mediante un fichero de bloqueo (``uv.lock``) y su compatibilidad con el estándar ``pyproject.toml`` actual.

Sistema gestor de base de datos: PostgreSQL 16 con pgvector

Para la persistencia se selecciona PostgreSQL en su versión 16, complementada con la extensión pgvector. PostgreSQL es un sistema gestor de bases de datos relacional, libre y de código abierto, con una madurez contrastada en entornos profesionales. La extensión pgvector permite almacenar y consultar embeddings semánticos dentro de la propia base de datos relacional, eliminando la necesidad de incorporar una base de datos vectorial especializada como componente adicional del sistema.

Orquestación de contenedores: Podman

Para la ejecución de PostgreSQL en entorno local se emplea Podman en su modo rootless, alternativa a Docker desarrollada por Red Hat. La compatibilidad de Podman con la sintaxis estándar de 'docker-compose.yml' permite que la configuración del proyecto sea ejecutable también con Docker, sin modificaciones, en máquinas que prefieran emplear esa alternativa.

Backend: Kotlin con framework Ktor

El servidor backend se desarrolla en Kotlin con el framework Ktor. Kotlin es un lenguaje moderno de tipado estático desarrollado por JetBrains, plenamente interoperable con la máquina virtual de Java (JVM). Sus principales virtudes técnicas son la verbosidad reducida frente a Java, el sistema robusto de seguridad frente a referencias nulas, y el soporte de primera clase a la programación asíncrona mediante corrutinas.

Ktor es el framework de aplicaciones web oficial de Kotlin, mantenido por JetBrains. Frente a alternativas más maduras como Spring Boot, Ktor presenta dos ventajas para el contexto del proyecto: una huella significativamente más liviana, lo que reduce el tiempo de arranque y el consumo de recursos, y un modelo de configuración explícito basado en composición de plugins, que permite incluir exclusivamente las funcionalidades necesarias y entender con facilidad qué está activo en el servidor.

Frontend: Kotlin Compose Multiplatform compilado a JavaScript

La aplicación web cliente se desarrolla en Compose Multiplatform, el framework de interfaz de usuario declarativo desarrollado por JetBrains para Kotlin, compilando específicamente al target de JavaScript.

La principal motivación para emplear Compose Multiplatform en lugar de un framework JavaScript clásico es la coherencia del stack: todo el código del proyecto (excepto el pipeline de datos en Python) se escribe en un único lenguaje, Kotlin. El módulo compartido 'shared' permite definir modelos de datos serializables que se emplean tanto en el backend como en el frontend, eliminando la necesidad de duplicar definiciones de tipos.

Modelo de embeddings: sentence-transformers

Para la generación de embeddings semánticos se emplea sentence-transformers, una biblioteca que ofrece modelos preentrenados optimizados para producir embeddings de oraciones. Se emplea el modelo all-mpnet-base-v2, que produce vectores de 768

dimensiones a partir de la arquitectura MPNet (Song et al., 2020). La elección de mpnet frente a alternativas más ligeras como all-MiniLM-L6-v2 responde a un compromiso donde la calidad de las recomendaciones prima sobre la velocidad de inferencia, dado que la generación de embeddings se ejecuta una sola vez de forma diferida.

Herramientas auxiliares

El proyecto se gestiona con Gradle como sistema de construcción para los módulos Kotlin, Git como sistema de control de versiones con repositorio remoto en GitHub, y JetBrains IntelliJ IDEA Ultimate como entorno integrado de desarrollo. La memoria se redacta inicialmente en formato Markdown y se exporta a PDF cumpliendo el formato exigido por la guía oficial del centro.

7.1 Planificar

7.1.1 Metodología de trabajo adoptada

Se adopta un enfoque iterativo e incremental inspirado en principios ágiles pero adaptado al contexto unipersonal del proyecto. Este enfoque presenta cuatro características definitorias.

- En primer lugar, el trabajo se organiza por bloques verticales: cada bloque produce un incremento funcional del sistema, en lugar de completar capa por capa de forma horizontal. Por ejemplo, el bloque dedicado al sistema de recomendación incluye desde la generación de los embeddings en el pipeline de datos hasta la pantalla del frontend que muestra las recomendaciones, pasando por la migración SQL que añade la columna vectorial y el endpoint REST que devuelve los resultados. Esto permite que al final de cada bloque exista un trozo de aplicación verificable y demostrable, en lugar de tener todas las capas a medio hacer.
- En segundo lugar, cada bloque se cierra con una fase de autocrítica documentada: tras completar un bloque, se revisa qué objetivos se han cumplido, qué problemas han aparecido y qué decisiones tomadas en bloques anteriores conviene revisar. Estas revisiones se vuelcan directamente en la memoria, lo que evita el problema clásico de redactar la documentación al final cuando ya se han olvidado las decisiones.
- En tercer lugar, el código se gestiona desde el primer día mediante control de versiones con Git, con un repositorio único para todo el proyecto y una política de **commits** atómicos por funcionalidad o corrección. El histórico de Git constituye una

fuentes complementarias de información para la memoria y permite reconstruir el orden cronológico real del desarrollo en caso necesario.

- En cuarto lugar, se aplica una estrategia de priorización por riesgo: los componentes técnicamente más inciertos (especialmente la integración con `pgvector` y la generación de embeddings) se abordan en bloques tempranos, no en bloques tardíos. Esta priorización busca evitar la situación habitual en proyectos académicos en la que un componente que se daba por sentado al inicio termina consumiendo más tiempo del previsto en la recta final.

7.1.2 Planificación temporal

El proyecto se desarrolla en un periodo aproximado de tres semanas, comprendido entre mediados de mayo y principios de junio. La planificación se organiza en nueve bloques de trabajo de duración variable.

A continuación se detalla cada bloque junto con su duración estimada.

ID	Bloque	Días
B1	Planificación del Proyecto, estructura y estudio de herramientas	7
B2	Análisis del dataset y diagnóstico de calidad	2
B3	Diseño de la base de datos	1
B4	Pipeline de limpieza y carga inicial	2
B5	Implementación del backend (capa REST)	3
B6	Implementación del frontend (pantallas base)	3
B8	Sistema de recomendación con embeddings	2
B9	Integración, pruebas y redacción final de la memoria	2

Figura 1. Bloques de trabajo y duración estimada de la planificación del proyecto.

La duración total estimada, considerando los solapamientos previstos, se sitúa en torno a veinte días naturales de trabajo. La planificación incluye además un margen de aproximadamente tres días para contingencias, dado que la experiencia previa en proyectos de complejidad similar indica que entre el 15% y el 20% del tiempo total se consume en imprevistos no contemplados en la planificación inicial.

7.1.3 Estrategia de priorización por riesgo

Tal como se ha indicado, los bloques técnicamente más inciertos se abordan en posiciones tempranas o intermedias de la planificación, no en posiciones finales. Concretamente, los tres riesgos principales identificados **a priori** son los siguientes.

- **Riesgo 1:** configuración e integración de pgvector. La extensión pgvector no forma parte de la instalación por defecto de PostgreSQL y su comportamiento en operaciones de carga masiva no ha sido validado previamente por el autor. Por este motivo, la primera operación que se realiza al inicio del bloque B5 (implementación del backend) es una carga sintética de prueba en una tabla con columna `vector`, con el objetivo de validar la cadena completa (instalación → migración → inserción → consulta por similitud) antes de invertir tiempo en otros componentes.
- **Riesgo 2:** tiempo de generación de los embeddings. La generación de embeddings para aproximadamente cuarenta y cinco mil sinopsis mediante un modelo ejecutado en CPU puede consumir un tiempo no despreciable. Se contempla por tanto la posibilidad de trabajar inicialmente con un subconjunto de mil películas seleccionadas (por ejemplo, las más populares según la métrica `popularity` del dataset original) para iterar rápidamente sobre el sistema de recomendación, y diferir la generación masiva de embeddings al final del proyecto, una vez validada la cadena completa.
- **Riesgo 3:** ajuste de Compose Multiplatform para el target JavaScript. El soporte de Compose Multiplatform para Web sigue evolucionando y las versiones recientes presentan diferencias en su sistema de empaquetado que pueden requerir ajustes específicos. Para mitigar este riesgo, el bloque B5 (backend) incluye desde el primer día una pantalla mínima en el frontend que verifica la comunicación cliente-servidor, garantizando que el camino crítico está abierto antes de iniciar el bloque B6, dedicado a las pantallas funcionales.

7.2 Buscar

7.2.1 Búsqueda y selección del dataset

El primer paso operativo del proyecto, una vez establecidos los objetivos y la arquitectura general, consistió en la búsqueda y selección de un dataset adecuado a los requisitos del trabajo. Los criterios de selección establecidos fueron los siguientes: licencia compatible con uso académico (preferentemente CC0 o equivalente, que minimiza las restricciones legales y administrativas), volumen suficiente para justificar técnicas de aprendizaje automático (al menos varios miles de registros), estructura relacional natural que permitiera modelar varias entidades con sus relaciones (en lugar de una única tabla plana), riqueza de información que justificara una webapp con múltiples vistas (en lugar

de un dashboard de modelo único), y presencia de contenido textual que permitiera aplicar técnicas de procesamiento del lenguaje natural y generación de embeddings semánticos.

Se seleccionó **The Movies Dataset** (autor: Rounak Banik, plataforma: Kaggle, licencia: CC0 1.0 Dominio Público) por cumplir simultáneamente todos los criterios establecidos: licencia plenamente libre, volumen significativo, estructura multi-tabla, riqueza temática que sustenta tanto la webapp como la memoria, y presencia de contenido textual rico en el campo overview (sinopsis) sobre el que aplicar técnicas de embeddings semánticos.

7.2.2 Estructura del dataset seleccionado

El dataset seleccionado se organiza en siete archivos CSV. A continuación se describen únicamente los cinco que entran dentro del alcance del proyecto. El archivo ratings.csv (26.024.289 filas) y su versión reducida ratings_small.csv quedan fuera del alcance, dado que el sistema de filtrado colaborativo está declarado como línea de trabajo futuro.

Archivo	Filas	Columnas	Función
movies_metadata.csv	45.466	24	Entidad central: una fila por película con sus metadatos
credits.csv	45.476	3	Equipo artístico (cast) y técnico (crew) por película
keywords.csv	46.419	2	Palabras clave temáticas por película
links.csv	45.843	3	Equivalencias entre identificadores de TMDb, IMDb y MovieLens
links_small.csv	9.125	3	Versión reducida de links.csv (descartada del proyecto)

Figura 2. Archivos del dataset seleccionado con su número de filas, columnas y función en el proyecto.

7.2.3 Metodología de inspección

Con el objetivo de obtener un diagnóstico fiable y reproducible de la calidad del dataset, se diseñó un script de inspección automatizado en Python que se ejecuta sobre los archivos CSV originales sin modificarlos. El script (recogido íntegramente en el repositorio del proyecto bajo la ruta python/scripts/01_inspect_raw.py) aplica tres niveles de comprobación sobre cada archivo.

El primer nivel, denominado known-issue checks, consiste en verificaciones específicas sobre problemas que se sospecha que existen en el dataset. Estas comprobaciones

derivan de documentación previa sobre el dataset y de la experiencia general de trabajar con datasets de origen abierto. Ejemplos incluyen la detección de valores cero encubiertos en variables económicas, la detección de duplicados por identificador y la validación de rangos plausibles para fechas.

El segundo nivel, denominado *broad sweep*, aplica genéricamente sobre todas las columnas un conjunto de estadísticas descriptivas estándar: tipos de datos detectados, valores ausentes por columna, percentiles para columnas numéricas, cardinalidad para columnas categóricas, longitudes para columnas de texto. Este nivel busca poner de manifiesto problemas que no se habían anticipado.

El tercer nivel, denominado *targeted checks*, aplica comprobaciones específicas dirigidas a anomalías conocidas en este dataset concreto. Incluye la detección de filas desplazadas mediante coherencia de tipos entre columnas, el análisis de caracteres no imprimibles y espacios anómalos en columnas de texto, la detección de duplicados semánticos por combinación de campos, y la validación de los campos anidados en formato JSON.

7.2.4 Resultados de la inspección por archivo

Sobre `movies_metadata.csv`

Es el archivo central del dataset y el que presenta mayor variedad de problemas de calidad. Sus 24 columnas se reparten entre identificadores (2), atributos numéricos (6), atributos categóricos (4), una fecha, atributos de texto libre (5) y campos con estructuras anidadas en formato JSON (6).

Columna	Ausentes	Porcentaje
<code>belongs_to_collection</code>	40.972	90,1%
<code>homepage</code>	37.684	82,9%
<code>tagline</code>	25.054	55,1%
<code>overview</code>	954	2,1%

Figura 3. Valores ausentes en las columnas principales del archivo `movies_metadata.csv`.

Por el contrario, los 954 valores ausentes en `overview` sí constituyen un problema relevante para el proyecto, dado que la sinopsis es el campo sobre el que se calculan los embeddings semánticos del sistema de recomendación. Las películas sin sinopsis no podrán recibir ni emitir recomendaciones basadas en contenido, lo que obliga a definir explícitamente una política de tratamiento (apartado 7.2.5).

Sobre credits.csv y keywords.csv

Ambos archivos presentan una estructura muy simple (identificador de película más uno o dos campos JSON) y no contienen valores ausentes. Sin embargo, ambos archivos contienen un volumen significativo de duplicados completos que conviene resolver al cargar:

- credits.csv: 37 filas duplicadas en su totalidad, afectando a 87 filas en relaciones de duplicación.
- keywords.csv: 987 filas duplicadas en su totalidad, afectando a 1.972 filas. Este volumen es desproporcionadamente alto y resulta uno de los hallazgos más llamativos de la inspección.

Sobre links.csv y links_small.csv

Ambos archivos comparten esquema (moviId, imdbId, tmdbId). El análisis comparativo del solapamiento entre los dos archivos arroja un resultado inesperado:

Conjunto	Cardinalidad
IDs en links_small.csv	9.125
IDs en links.csv	45.843
IDs en ambos	9.115
Solo en links_small.csv	10
Solo en links.csv	36.728

Figura 4. Solapamiento entre los archivos links.csv y links_small.csv.

La existencia de 10 identificadores presentes únicamente en la versión reducida demuestra que links_small.csv no es un subconjunto estricto de links.csv sino un archivo generado con criterios parcialmente independientes. Esta observación lleva a la decisión de utilizar exclusivamente links.csv en el proyecto y descartar la versión reducida, evitando la duplicidad y la complejidad de tratar las inconsistencias.

7.2.5 Catálogo de problemas identificados

La inspección identificó diez problemas de calidad catalogados como P1 a P10. La siguiente tabla resume cada problema, su impacto y la estrategia de tratamiento adoptada.

ID	Problema	Filas afectadas	Estrategia
P1	Filas desplazadas (columnas corridas por comas no escapadas)	3	Eliminar
P2	Ceros encubiertos en budget/revenue (ausencia, no valor real)	36.573 / 38.052	Sustituir por NaN
P3	vote_average=0 sin votos (valor espurio)	2.998	NaN condicional a vote_count=0
P4	Runtime=0 o >300 min (ausencia u outlier)	1.558	NaN para 0 y >300
P5	Películas sin géneros asignados	2.442	Conservar sin etiquetar
P6	Dobles espacios, non-breaking spaces en textos	1.348+	Normalizar whitespace
P7	Sinopsis placeholder ("No overview found", "N/A")	141	Sustituir por NaN
P8	Duplicados completos en credits (37) y keywords (987)	1.024	Deduplicar
P9	Duplicados por imdb_id en movies_metadata	79	Deduplicar conservando primera ocurrencia
P10	Fechas anteriores a 1880	2	Conservar (posible proto-cine)

Figura 5. Catálogo de problemas de calidad identificados en el dataset, con filas afectadas y estrategia de tratamiento.

Los problemas P2, P3, P6 y P7 tienen impacto directo sobre el sistema de recomendación: los ceros encubiertos distorsionan estadísticas, y el texto sin normalizar o con placeholders degrada la calidad de los embeddings semánticos. Las estrategias priorizan la conservación del catálogo completo, marcando los campos problemáticos como ausentes en lugar de eliminar las películas afectadas.

7.2.6 Decisiones de diseño consolidadas

Sobre el modelado de la base de datos, se confirma la viabilidad de un esquema relacional normalizado con movies como entidad central. La integridad referencial impecable entre archivos (cero huérfanos en credits y keywords) permite definir claves foráneas estrictas sin necesidad de tolerar valores no encontrados.

Sobre el pipeline de limpieza, se establece la secuencia operativa siguiente: (1) saneamiento estructural eliminando las tres filas desplazadas y casteando los tipos de datos; (2) marcado como ausentes de los valores cero encubiertos en budget, revenue, vote_average (condicional a vote_count) y runtime; (3) normalización del texto en columnas de sinopsis y eslogan; (4) deduplicación de los tres archivos principales; (5) tratamiento de placeholders en overview; (6) etiquetado especial de las películas sin género.

Sobre el sistema de recomendación, se confirma la necesidad de normalizar el texto antes del cálculo de embeddings y de excluir las películas con sinopsis ausente o placeholder. Las películas sin sinopsis quedarán visibles en la webapp pero no aparecerán como candidatas en las recomendaciones, lo que se documentará explícitamente en la interfaz.

Sobre los archivos a cargar, se confirma el uso de movies_metadata.csv, credits.csv, keywords.csv y links.csv. Se descarta links_small.csv por no constituir un subconjunto estricto del completo, y se mantiene fuera del alcance ratings.csv por estar el filtrado colaborativo declarado como línea de trabajo futuro.

7.3 Ejecutar

7.3.1 Limpieza y transformación de datos

Arquitectura del pipeline. Se optó por una arquitectura de scripts independientes orquestados externamente, frente a la alternativa de un único script monolítico. Cada script de limpieza lee un archivo CSV original de data/raw/, aplica sus transformaciones y escribe el resultado en data/processed/. Esta independencia tiene tres ventajas: permite ejecutar cualquier paso de forma aislada durante el desarrollo y la depuración, facilita la adición futura de nuevos archivos sin modificar los existentes, y produce una salida intermedia inspeccionable para cada paso (los archivos procesados pueden verificarse manualmente antes de cargar a la base de datos).

Herramientas. Se seleccionó pandas como biblioteca de manipulación de datos por ser el estándar de facto para procesamiento de CSV en Python y por ofrecer las operaciones necesarias (filtrado, casting, deduplicación, manipulación de texto) de forma expresiva y eficiente para datasets de este tamaño (decenas de miles de filas). Se descartaron alternativas como Polars (más rápida en datasets de millones de filas, pero innecesaria aquí y con una API menos documentada en el contexto académico) y procesamiento línea

a línea con la biblioteca estándar csv (demasiado verboso para las transformaciones requeridas).

Las funciones de limpieza reutilizables se centralizaron en un módulo python/src/cleaning.py para evitar duplicación entre scripts. Este módulo expone funciones puras que reciben un DataFrame, lo transforman y devuelven un DataFrame nuevo, siguiendo el patrón de pipeline funcional. Cada función incluye logging del número de filas afectadas antes y después de la transformación, lo que permite auditar el impacto de cada paso sin inspeccionar manualmente los archivos de salida.

Módulo de funciones de limpieza (python/src/cleaning.py). Se implementaron diecisiete funciones de limpieza reutilizables, organizadas por categoría:

- **Saneamiento estructural** (drop_shifted_rows, cast_types, parse_release_date).
- **Tratamiento de valores encubiertos** (zeros_to_nan, vote_average_to_nan_when_no_votes, clip_runtime, mask_suspicious_budget, mask_unreliable_ratings)
- **Normalización de texto** (normalize_whitespace, replace_placeholders_with_nan, mask_short_overviews).
- **Deduplicación** (drop_full_duplicates, drop_duplicates_by_key).
- **Filtrado** (filter_adult)
- **Tratamiento de estado** (fill_status_unknown). Cada función registra mediante el logger el número de filas modificadas, eliminadas o marcadas como ausentes.

Script de limpieza de películas (**02_clean_movies.py**). Es el script más complejo del pipeline por la variedad de problemas que debe tratar. La secuencia de transformaciones sigue un orden estricto: primero se eliminan las tres filas desplazadas (P1), después se convierten los tipos de datos, se parsea la fecha de estreno, se reemplazan los ceros encubiertos por valores ausentes (P2, P3, P4), se normalizan los textos (P6), se eliminan los placeholders de sinopsis (P7), se marcan los presupuestos sospechosos (valores redondos improbables), se filtran las películas adultas, y finalmente se deduplican por identificador (P9). El orden no es arbitrario: la eliminación de filas desplazadas debe preceder al casting de tipos (las filas desplazadas causarían errores de conversión), y la deduplicación debe ser posterior a la normalización de texto (para que las comparaciones sean coherentes).

Scripts de limpieza de `credits.csv`, `keywords.csv` y `links.csv`. Los tres scripts son significativamente más simples que el de películas. La limpieza de `credits.csv` aplica dos pases de deduplicación: uno por filas completas (P8, 37 duplicados) y otro por identificador (11 filas adicionales con mismo ID pero contenido distinto). La limpieza de `keywords.csv` aplica un único pase de deduplicación completa (P8, 987 duplicados). La limpieza de `links.csv` elimina filas sin identificador TMDb, convierte `tmdbId` de float a entero, y deduplica por identificador TMDb para garantizar la restricción de unicidad requerida por la clave foránea en la base de datos.

Orquestador (**`06_run_pipeline.py`**). El orquestador ejecuta los cuatro scripts de limpieza en secuencia mediante `subprocess.run`, usando el mismo intérprete Python que lo ejecuta (garantizando que se respeta el entorno virtual). Si cualquier script falla (código de salida distinto de cero), el pipeline aborta inmediatamente. Al finalizar, imprime un resumen con el nombre de cada script, su estado (OK/FAIL) y su duración. La ejecución completa del pipeline sobre el dataset original se completó en aproximadamente tres segundos.

Auditoría secundaria. Durante la implementación del script de películas se descubrieron cinco problemas adicionales no detectados en la inspección inicial (denominados P11 a P17 en los comentarios del código): duplicados por identificador en `credits.csv` con contenido discordante (P11), duplicados por `tmdbId` en `links.csv` (P12), presupuestos con valores sospechosamente redondos (P13), películas con cero votos pero valoración distinta de cero (P14), y sinopsis excesivamente cortas que no aportan valor semántico (P15). Estos problemas se integraron en el pipeline sin alterar su estructura, añadiendo funciones de limpieza adicionales al módulo `cleaning.py`. La auditoría secundaria confirma el valor del enfoque iterativo: una primera pasada identifica los problemas evidentes, la implementación revela los sutiles.

7.3.2 Diseño de la base de datos y carga de datos

PostgreSQL con `pgvector` frente a alternativas. La elección de PostgreSQL como gestor de base de datos se realizó en la fase de planificación (apartado 7.1) por dos motivos. Primero, PostgreSQL es la base de datos relacional de código abierto más madura y mejor documentada, con soporte nativo para tipos de datos avanzados (arrays, JSON, fechas con rango) que se adaptan naturalmente a la estructura del dataset. Segundo, la extensión `pgvector` permite almacenar vectores de alta dimensionalidad y realizar búsquedas de vecinos más cercanos directamente en SQL, sin necesidad de un servicio vectorial externo. Esta integración reduce la complejidad de la arquitectura del sistema a

un único gestor de datos que sirve tanto las consultas relacionales del catálogo como las consultas vectoriales del sistema de recomendación.

Esquema normalizado frente a desnormalizado. El esquema sigue la tercera forma normal (3NF) con tablas de unión para las relaciones muchos-a-muchos (película-género, película-compañía, etc.). Se descartó un esquema desnormalizado (una única tabla ancha con arrays de géneros, JSON de créditos, etc.) por dos motivos: primero, las consultas de filtrado por género, por compañía o por persona son significativamente más eficientes cuando operan sobre tablas normalizadas con índices; segundo, la normalización refleja la estructura natural del dominio y produce un esquema auto-documentado que un evaluador puede comprender inspeccionando únicamente las tablas y sus relaciones.

Para el endpoint de detalle (GET /movies/{id}), donde se necesita toda la información agregada en una sola respuesta, se creó una vista materializable (v_movie_details) que une todas las tablas mediante subconsultas correlacionadas y funciones de agregación (ARRAY_AGG, JSON_AGG). De este modo, el backend ejecuta una única consulta en lugar de una por cada relación, sin sacrificar la normalización del esquema subyacente.

Infraestructura de contenedores (docker-compose.yml). Se configuró un servicio postgres basado en la imagen pgvector/pgvector:pg16, que incluye PostgreSQL 16 con la extensión pgvector preinstalada. El contenedor expone el puerto 5432, monta un volumen persistente postgres_data para los datos, e incluye un healthcheck que verifica la disponibilidad del servicio mediante pg_isready. Las credenciales se leen de la variable de entorno DB_PASSWORD definida en el fichero .env, con un valor por defecto para simplificar la primera ejecución.

La migración **2026052500_init_extensions.sql** activa las extensiones pgvector (para el tipo vector y los operadores de distancia) y unaccent (para búsqueda de texto sin sensibilidad a acentos), creando además una función immutable_unaccent que permite su uso en índices funcionales (la función unaccent estándar de PostgreSQL está marcada como STABLE, no como IMMUTABLE, lo que impide su uso directo en expresiones de índice).

La migración **2026052501_create_movies_and_collections.sql** crea la tabla collections (franquicias como "Toy Story Collection") y la tabla central movies con veinticuatro columnas que replican fielmente los campos del CSV procesado: identificadores, metadatos textuales (title, original_title, overview, tagline), atributos numéricos (budget, revenue, runtime, popularity, vote_average, vote_count), fecha de estreno, estado

(Released, Post Production, etc.), indicador de contenido adulto, rutas de póster, y una clave foránea a collections.

La migración **2026052502_create_people_and_credits.sql** crea la tabla people (actores, directores, guionistas) y las tablas de unión movie_cast (con campos character y order para el personaje interpretado y el orden de aparición en créditos) y movie_crew (con campos job y department).

La migración **2026052503_create_categories.sql** crea cinco tablas maestras (genres, production_companies, production_countries, spoken_languages, keywords) y sus correspondientes cinco tablas de unión con movies. Todas las tablas maestras tienen estructura idéntica (id, name) y todas las tablas de unión tienen estructura idéntica (movie_id, entity_id, clave primaria compuesta).

La migración **2026052504_create_external_ids.sql** crea la tabla external_ids que mapea cada película a sus identificadores en MovieLens (movielens_id) e IMDb (imdb_id).

Sistema de migraciones (**07_migrate.py**). El script conecta a PostgreSQL, crea la tabla schema_migrations si no existe (con columnas filename, applied_at y duration_ms), lee todos los ficheros .sql del directorio migrations/, compara sus nombres con los ya registrados en schema_migrations, y aplica los pendientes en orden alfabético. Cada migración se ejecuta dentro de una transacción; si falla, la transacción se revierte y el script aborta, dejando la base de datos en un estado consistente. Tras cada migración exitosa, se registra el nombre del fichero y la duración en milisegundos. La idempotencia está garantizada: ejecutar el script dos veces consecutivas no tiene efecto (la segunda ejecución no encuentra migraciones pendientes).

Scripts de carga de datos. Los cuatro scripts de carga (**08_load_movies.py**, **09_load_categories.py**, **10_load_credits.py**, **11_load_links.py**) siguen un patrón común: leer el CSV procesado con pandas, parsear los campos JSON embebidos cuando es necesario, acumular las entidades únicas y las filas de unión, e insertarlas en la base de datos. Las entidades maestras (colecciones, géneros, compañías, personas) se insertan con ON CONFLICT DO NOTHING para garantizar la idempotencia. Las filas de unión y las películas se cargan mediante la función COPY de PostgreSQL (expuesta como cursor.copy en psycopg3), que es entre diez y cien veces más rápida que inserts individuales para volúmenes de miles de filas.

El script de categorías (**09_load_categories.py**) es el más complejo de los cuatro: procesa cinco familias de categorías (géneros, compañías productoras, países de

producción, idiomas hablados y palabras clave) con un patrón uniforme pero datos de origen diversos (cuatro familias provienen de columnas JSON en movies.csv, la quinta proviene de keywords.csv).

El orquestador (**12_run_load_pipeline.py**) ejecuta los cuatro scripts en secuencia con el mismo mecanismo que el orquestador de limpieza: ejecución con subprocess.run, temporización de cada paso, aborto inmediato si alguno falla. La carga completa del dataset (cuarenta y cinco mil películas, más de un millón de filas de unión, trescientas mil personas) se completa en aproximadamente quince segundos.

Vistas e índices. Tras la carga de datos, tres migraciones adicionales optimizan las consultas del backend. La vista **v_movie_details** agrega toda la información de una película en una sola fila, incluyendo géneros, países, idiomas, compañías productoras, palabras clave, reparto y equipo técnico como arrays o JSON, eliminando la necesidad de múltiples joins en el código del backend. Los índices de listado (**2026052601**) cubren las columnas de ordenación y filtrado más frecuentes (popularity, vote_average, release_date, title, original_language). Los índices de búsqueda (**2026052602**) son funcionales, aplicando la misma normalización que las consultas de búsqueda del backend (lower, unaccent, reemplazo de guiones por espacios) para que el planificador de PostgreSQL pueda utilizarlos.

Parseo de campos JSON embebidos en CSV. Los archivos movies_metadata.csv y credits.csv contienen columnas cuyo valor es un string que representa una lista JSON (por ejemplo, la columna genres contiene valores como [{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}]). Sin embargo, estos strings no están codificados como JSON estándar: utilizan comillas simples en lugar de dobles, y los valores booleanos aparecen como True/False (notación Python) en lugar de true/false (notación JSON). El parseo con json.loads() falla en estos casos. La solución implementada utiliza ast.literal_eval() de la biblioteca estándar de Python, que interpreta correctamente la notación de literales Python, envuelto en un manejador de errores que devuelve una lista vacía para valores no parseables. Esta función (parse_python_literal en el módulo loading.py) se reutiliza en todos los scripts de carga que procesan campos JSON.

Función unaccent no utilizable en índices. PostgreSQL clasifica la función unaccent como STABLE (su resultado puede variar entre transacciones), lo que impide su uso en expresiones de índice (que requieren funciones IMMUTABLE). Dado que en la práctica la función unaccent es determinista (su resultado para una entrada dada no cambia nunca), se creó un wrapper immutable_unaccent declarado como IMMUTABLE que simplemente

invoca unaccent internamente. Esta técnica es una práctica documentada en la comunidad de PostgreSQL para resolver esta limitación. El wrapper se crea en la primera migración para estar disponible en las migraciones posteriores de índices de búsqueda.

Orden de carga y claves foráneas. La tabla movies referencia a collections mediante una clave foránea. Esto obliga a cargar las colecciones antes que las películas, lo que a su vez obliga a extraer las colecciones del mismo archivo CSV que contiene las películas (movies.csv), dado que no existe un archivo separado de colecciones en el dataset original. El script **08_load_movies.py** resuelve esto con un primer pase que extrae y deduplica las colecciones (parseando el campo JSON belongs_to_collection), las inserta en la tabla collections, y un segundo pase que carga las películas con la referencia ya establecida. El mismo patrón se repite a menor escala en los scripts de categorías (las entidades maestras deben existir antes de que las tablas de unión puedan referenciarlas).

7.3.3 Desarrollo del backend

El backend se descompuso en seis sub-pasos: configuración inicial con endpoint de salud, acceso a base de datos con pool de conexiones, endpoint de detalle, endpoint de listado paginado con filtros, endpoint de búsqueda por texto, y suite de pruebas funcionales. Tres compromisos arquitectónicos se respetan a lo largo del bloque: separación estricta en tres capas (rutas, servicios, repositorios), módulo compartido para DTOs, y contrato de errores estructurado con códigos máquina (movie_not_found, invalid_id, etc.).

El frontend del proyecto se desarrolla en Kotlin sobre Compose Multiplatform compilado a JavaScript. Esta decisión, motivada por la posibilidad de compartir código entre cliente y servidor mediante un módulo común, restringe la elección del backend a tecnologías que puedan cooperar con un módulo Kotlin compartido. La opción natural es desarrollar también el servidor en Kotlin, lo que garantiza coherencia tecnológica y permite reutilizar definiciones de tipos sin intermediarios.

Dentro del ecosistema Kotlin de servidor, las dos opciones principales son Spring Boot y Ktor. Spring Boot es el framework dominante en el ecosistema Java y dispone de soporte oficial para Kotlin, pero su modelo de configuración basado en convenciones implícitas, autoconfiguración por anotaciones y un ecosistema masivo de starters resulta excesivo para el alcance acotado del presente proyecto. Ktor, mantenido por JetBrains (la misma empresa que desarrolla Kotlin), adopta un modelo radicalmente distinto: la configuración del servidor es explícita y se construye mediante composición de plugins en código, sin

descubrimiento automático ni anotaciones. Esto produce arranques más rápidos, una huella de memoria menor y, sobre todo, código en el que es trivial identificar qué funcionalidades están activas. Para un proyecto de tamaño TFG, la simplicidad y transparencia de Ktor priman sobre la madurez ecosistémica de Spring Boot.

El acceso a la base de datos se resuelve mediante el conector JDBC oficial de PostgreSQL (postgresql:42.7.4) gestionado por un pool de conexiones HikariCP (com.zaxxer:HikariCP:6.2.1). Sobre esta combinación se descarta deliberadamente el uso de un ORM (Object-Relational Mapper) como Exposed (de JetBrains) o Hibernate. Esta decisión merece justificación explícita, pues podría parecer una elección regresiva en un contexto donde los ORMs son práctica habitual.

Tres razones la motivan. Primera, las consultas que el backend necesita ejecutar son fundamentalmente de lectura, con filtros dinámicos y agregaciones a través de varias tablas relacionales. Este patrón se expresa de forma más clara y eficiente en SQL directo que mediante el lenguaje de queries de un ORM, que suele convertirse en una abstracción que oculta más de lo que aclara cuando la consulta es compleja. Segunda, la vista de detalle de una película (GET /movies/{id}) requiere agregar datos de nueve tablas relacionales en una única respuesta JSON, lo que se resuelve elegantemente mediante una vista SQL con funciones de agregación (ARRAY_AGG, JSON_AGG) que un ORM no genera de forma natural. Tercera, JDBC y SQL son competencias del propio currículo del ciclo formativo de DAM, mientras que el aprendizaje de un ORM concreto reduciría el tiempo dedicado a la lógica del proyecto sin aportar valor académico equivalente.

La serialización de respuestas y peticiones se realiza mediante `kotlinx.serialization` con su backend JSON, biblioteca oficial mantenida por JetBrains. Frente a alternativas como Jackson o Gson, `kotlinx.serialization` presenta dos ventajas para este proyecto: opera en tiempo de compilación mediante generación de código, lo que evita el uso de reflexión en tiempo de ejecución y produce mejor rendimiento; y está diseñada desde el principio con soporte multiplataforma, lo que permite que las mismas clases serializables del módulo `:shared` se utilicen tanto en el servidor (JVM) como en el frontend (JavaScript) sin adaptaciones.

Para la suite de pruebas funcionales se selecciona el componente `ktor-server-test-host`, que constituye la solución oficial para testar aplicaciones Ktor. Su característica diferencial es que no abre un puerto TCP real durante la ejecución de los tests: el cliente HTTP del test se comunica con el servidor en memoria a través de un canal interno, lo que reduce el tiempo de ejecución de cada prueba de varios segundos (los necesarios para arrancar un

servidor real) a milisegundos. Combinado con JUnit 5 como framework de aserciones y con la pequeña biblioteca kotlin-test que aporta funciones idiomáticas en Kotlin, esto permite ejecutar las veintitrés pruebas funcionales del bloque en menos de dos segundos.

Endpoint de detalle (GET /movies/{id}). Devuelve toda la información de una película (datos propios, géneros, países, idiomas, productoras, palabras clave, reparto y equipo técnico) consultando la vista SQL v_movie_details, que agrega datos de nueve tablas mediante ARRAY_AGG y JSON_AGG en una sola fila. Los DTOs (MovieDetails, CastMember, CrewMember, etc.) viven en el módulo :shared.

Endpoint de listado (GET /movies). Listado paginado con filtros combinables (género, año, idioma, mínimo de votos), ordenación configurable y paginación. La consulta SQL se construye dinámicamente con prepared statements. Se usa EXISTS para el filtro por género (mejor rendimiento que JOIN+DISTINCT), tiebreaker por ID para paginación reproducible, y NULLS LAST en ordenaciones descendentes. Un minVoteCount por defecto de 10 se aplica automáticamente al ordenar por valoración, evitando que películas con un solo voto perfecto dominen los resultados.

Endpoint de búsqueda (GET /movies/search?q=...). Búsqueda por título insensible a mayúsculas, diacríticos y separadores (guion/espacio) mediante ILIKE sobre una expresión normalizada replace(lower(immutable_unaccent(col)), '-', ' '). Un sistema de puntuación prioriza coincidencias exactas (100 puntos) sobre parciales (10 puntos). Índices funcionales contruidos sobre la misma expresión normalizadora garantizan rendimiento.

Suite de pruebas. Treinta pruebas de integración con ktor-server-test-host (sin socket TCP real) contra la base de datos de desarrollo. Cubren casos de éxito, errores de validación, recursos inexistentes y parámetros fuera de rango. Tiempo total: menos de diez segundos.

Problemas encontrados

- Iteración de versiones del ecosistema Kotlin. El sub-paso de configuración inicial requirió seis ciclos de modificación del build.gradle.kts. La tabla de compatibilidad de Ktor no incluía las versiones más recientes del compilador, y la combinación sugerida por el wizard de IntelliJ era inestable. La estrategia de priorización por riesgo se reveló acertada: este problema se abordó antes de invertir en lógica de negocio.

- Normalización de separadores en búsqueda. Durante verificación empírica se descubrió que spider man no encontraba Spider-Man (guion vs espacio). Se amplió la expresión de normalización y se reconstruyeron los índices.

7.3.4 Desarrollo del sistema de recomendación

El bloque se descompuso en cinco sub-pasos: evolución del esquema de embeddings (columnas `model_name` y `source_text_hash` para idempotencia), script de generación de embeddings, índice HNSW, endpoint de películas similares, y suite de pruebas.

Decisión sobre el modelo concreto. Dentro de la familia sentence-transformers existen decenas de modelos pre-entrenados con compromisos distintos entre calidad, dimensión del vector resultante y velocidad de inferencia. Se evalúan dos candidatos. El primero, `all-MiniLM-L6-v2`, es un modelo ligero de seis capas que produce vectores de 384 dimensiones; su principal virtud es la velocidad (aproximadamente noventa y seis películas por segundo sobre la CPU del entorno de desarrollo) y la modestia de su consumo de memoria. El segundo, `all-mpnet-base-v2`, es un modelo de doce capas basado en la arquitectura MPNet (Song et al., 2020) que produce vectores de 768 dimensiones; su principal virtud es la mayor calidad de los embeddings generados, según los benchmarks oficiales publicados por la propia biblioteca sentence-transformers, a costa de un coste computacional notablemente superior.

La selección final recae en all-mpnet-base-v2. La justificación se apoya en dos consideraciones. Primera, la fase de generación de embeddings se ejecuta una sola vez de forma diferida, no en línea durante el servicio de peticiones; en consecuencia, el coste computacional adicional (un factor de tres a cinco respecto al modelo ligero) se paga una única vez y no impacta sobre la latencia percibida por el usuario final. Segunda, la fase de consulta se realiza sobre los vectores ya almacenados mediante el índice HNSW, cuyo coste es prácticamente independiente de la dimensión del vector (logarítmico en el número de vectores, no en la dimensión); en consecuencia, la mayor dimensión del modelo más pesado no penaliza significativamente la velocidad del endpoint. Estas dos consideraciones inclinan la balanza hacia la calidad frente a la velocidad. El sub-apartado 7.3.4.4 documenta no obstante una iteración intermedia: el desarrollo se inició con el modelo ligero, se evaluaron empíricamente las recomendaciones obtenidas, se detectaron limitaciones en la captura de relaciones temáticas implícitas, y se migró al modelo pesado, obteniéndose mejoras notables en dos de los tres casos de prueba empleados.

Composición del texto fuente. El embedding de cada película se calcula a partir de un texto compuesto, no directamente de la sinopsis. La composición concatena, separados por puntos, cuatro elementos: el título de la película, la lista de géneros, la lista de palabras clave y la sinopsis. La motivación de incluir géneros y palabras clave, no únicamente la sinopsis, responde a una observación empírica realizada durante el desarrollo: cuando el embedding se calcula únicamente sobre la sinopsis, el modelo tiende a anclarse a las palabras superficiales del texto (por ejemplo, "fight" o "club" en el caso de Fight Club) en detrimento de conceptos temáticos que la sinopsis no enuncia explícitamente. La inclusión de palabras clave curadas por la comunidad de TMDb (identity crisis, dual personality, consumerism, dream within dream...) introduce explícitamente esa información temática y mejora la calidad de las recomendaciones, según se documenta en el sub-apartado 7.3.4.3. La lista de palabras clave se limita a las veinte primeras para no desplazar la sinopsis fuera de la ventana de doscientos cincuenta y seis tokens admitida por el modelo.

Decisión sobre el tipo de índice. PostgreSQL, a través de la extensión pgvector, ofrece dos tipos de índice especializados en búsqueda aproximada de vecinos más cercanos: IVFFlat (basado en Inverted File con compresión plana, particionado del espacio mediante clustering) y HNSW (basado en Hierarchical Navigable Small World, grafo navegable de múltiples capas). IVFFlat requiere especificar el número de particiones como parámetro de construcción, valor cuya elección óptima depende del tamaño del dataset y debe revisarse si éste crece significativamente. HNSW no presenta este parámetro de dependencia con el tamaño del dataset, lo que simplifica el mantenimiento del esquema a lo largo del tiempo. En cuanto a rendimiento, la documentación oficial de pgvector y la literatura especializada (Malkov y Yashunin, 2018) reportan consistentemente que HNSW alcanza mejor recall (proporción de vecinos reales recuperados respecto a los teóricos) que IVFFlat, a costa de mayor tiempo de construcción y mayor consumo de memoria. Para el orden de magnitud del catálogo del presente proyecto (cuarenta y cuatro mil vectores), tanto el tiempo de construcción del índice (medido empíricamente en aproximadamente cuarenta y seis segundos) como el consumo de memoria (ciento setenta y tres megabytes en disco) resultan asumibles, lo que decanta la elección hacia HNSW.

Decisión sobre la métrica de distancia. pgvector soporta tres operadores de distancia: distancia coseno ($\cos$), distancia euclídea (L2) y producto interno negativo (dot). Para embeddings de texto producidos por modelos de la familia sentence-transformers la

métrica recomendada por los propios autores es la distancia coseno. Esta métrica es invariante a la magnitud del vector, lo que evita que diferencias de longitud del texto fuente se traduzcan en diferencias artificiales de distancia. Adicionalmente, los embeddings producidos por estos modelos están aproximadamente normalizados, por lo que las tres métricas producirían rankings similares; la diferencia es matemáticamente menor pero la distancia coseno es la convención más extendida en la literatura y la que mejor documenta su comportamiento. El índice HNSW se construye con la familia de operadores correspondiente (`vector_cosine_ops`).

Generación de embeddings. El **script `13_generate_embeddings.py`** construye para cada película un texto fuente combinando título, géneros, palabras clave y sinopsis, calcula el embedding con `mpnet`, y lo persiste con `ON CONFLICT` para idempotencia.

Índice HNSW. Se construyó un índice HNSW con parámetros `m=16`, `ef_construction=64` sobre los vectores de 768 dimensiones, usando la métrica de distancia coseno. El índice ocupa 173 MB y se construyó en 46 segundos.

Endpoint `GET /movies/{id}/similar`. Devuelve las películas más cercanas por distancia coseno. Parámetros: `limit` (1-50, defecto 10), `minVoteCount` (opcional). Respuesta: lista de `SimilarMovieItem` (wrapping `MovieSummary` + puntuación de similitud). Registrado antes de `/id` para evitar colisión de rutas. Tiempo de respuesta: ~16 ms en caliente, ~300 ms en frío.

Iteración del modelo de embeddings. Se evaluaron empíricamente tres configuraciones con tres películas de referencia (`Fight Club`, `Inception`, `Toy Story`). `MiniLM-L6-v2` con solo título+sinopsis capturaba mal los temas; la adición de géneros y keywords mejoró `Inception` y `Toy Story` pero empeoró `Fight Club`. El cambio a `mpnet` (768 dimensiones) mejoró la calidad global. La limitación de `Fight Club` se identificó como inherente al texto fuente: la sinopsis de TMDb describe la trama literalmente sin mencionar los temas subyacentes (identidad, consumismo).

Migración destructiva de dimensiones. El cambio de 384 a 768 dimensiones obligó a truncar la columna de embeddings, regenerar todos los vectores y reconstruir el índice HNSW. El nombre del fichero de migración contiene un typo (786 en lugar de 768), preservado por integridad del historial de migraciones.

7.3.5 Desarrollo de la aplicación cliente

El frontend se descompuso en ocho sub-pasos: centralización de versiones en Gradle Version Catalog y conversión de `:shared` a Kotlin Multiplatform (B7.1), creación del módulo

:composeApp (B7.2), cliente HTTP tipado (B7.3), pantalla de listado con filtros e infinite scroll (B7.4), pantalla de detalle con recomendaciones (B7.5), búsqueda con autocompletado (B7.6), alternancia de tema claro/oscuro (B7.7), y servir frontend desde Ktor (B7.8).

Compose Multiplatform para la Web frente a alternativas. La decisión principal del bloque fue la elección de Compose HTML como tecnología de interfaz. La familia Compose Multiplatform ofrece dos modos de ejecución para la web: Compose HTML (también denominado Compose for Web DOM) y Compose for Canvas (anteriormente Compose for Web Canvas, basado en Skia/Skiko). Compose for Canvas dibuja la interfaz sobre un elemento `<canvas>` de HTML5 utilizando el mismo motor de renderizado que Compose para escritorio y Android, lo que permite compartir código visual al cien por cien entre plataformas; sin embargo, el contenido renderizado en un canvas es opaco para el navegador: el texto no es seleccionable, las herramientas de accesibilidad no lo detectan, los buscadores no lo indexan y las herramientas de desarrollo no permiten inspeccionar elementos individuales. Compose HTML, por el contrario, genera elementos DOM reales (`<div>`, `<span>`, `<img>`, `<input>`), lo que preserva la accesibilidad, la inspección en DevTools, la selección de texto y el comportamiento natural del navegador (scroll, eventos de foco, formularios), a cambio de requerir estilos CSS explícitos en lugar de los modificadores de Compose Material.

Para un proyecto académico que necesita demostrar la interfaz ante un tribunal y cuya interfaz se compone de texto, imágenes y controles estándar, Compose HTML es la opción más defendible: el bundle producido es significativamente más pequeño (1,19 MiB frente a los cinco o seis del modo Canvas por la inclusión de Skiko), la experiencia del usuario es indistinguible de una aplicación web convencional, y las herramientas de depuración del navegador son plenamente operativas.

Se descartó también la opción de utilizar React, Vue o Angular para el frontend. Estas tecnologías habrían introducido un lenguaje adicional (TypeScript o JavaScript), habrían obligado a duplicar o generar los DTOs del backend, y habrían impedido demostrar una de las propiedades diferenciales de Kotlin Multiplatform: la capacidad de compartir modelos de datos entre servidor y cliente compilando el mismo código Kotlin a JVM y a JavaScript.

Navegación sin biblioteca externa. En aplicaciones web monopágina (SPA) es habitual recurrir a bibliotecas de enrutamiento que gestionan el mapeo entre la URL del navegador y la pantalla visible. En el ecosistema de Compose Multiplatform la biblioteca más

referenciada es Voyager; sin embargo, al momento del desarrollo Voyager no soportaba el target Compose HTML (su compatibilidad se limita a Compose for Canvas, Android, iOS y escritorio). Ante esta incompatibilidad se optó por implementar un enrutamiento manual basado en fragmentos de URL (hash routing), donde la parte de la URL posterior al carácter # identifica la pantalla activa (/#/movies, /#/movie/550, /#/search/matrix). El hash nunca se envía al servidor, lo que elimina la necesidad de configurar un catch-all o fallback en Ktor para devolver index.html ante rutas desconocidas. El estado de la ruta se gestiona en un objeto singleton AppState con una propiedad observable currentRoute que Compose observa automáticamente para decidir qué pantalla renderizar; la integración con la API History del navegador (pushState, evento popstate) permite que los botones de avance y retroceso funcionen de forma nativa. El proyecto referente de la consultoría externa empleaba el mismo patrón con quince o más pantallas, lo que validó su escalabilidad para proyectos de mayor envergadura.

CSS custom properties frente a constantes Kotlin. El sistema de estilado se diseñó en torno a variables CSS declaradas en la pseudoclase :root y consumidas en todo el árbol de estilos mediante var(--nombre). Este enfoque se eligió sobre la alternativa de definir los colores como constantes Kotlin y pasarlos como parámetros a los Composables por tres motivos: primero, la alternancia entre modo oscuro y modo claro se implementa como la adición o eliminación de una clase CSS en el elemento <body>, sin recomposición del árbol de Compose; segundo, las variables CSS son visibles y editables en tiempo real en las herramientas de desarrollo del navegador, lo que acelera enormemente la iteración visual durante el desarrollo; y tercero, los componentes individuales referencian tokens semánticos (--text-primary, --bg-secondary, --accent) en lugar de colores literales, estableciendo un contrato de diseño que facilita cambios globales de paleta sin modificar ningún componente.

Infraestructura Gradle y módulo compartido (B7.1-B7.2). Se centralizaron las versiones de dependencias en gradle/libs.versions.toml. El módulo :shared se convirtió a Kotlin Multiplatform con targets jvm() y js(IR) { browser() }, trasladando los ocho DTOs a src/commonMain/kotlin/ mediante git mv. Se creó el módulo :composeApp con Compose HTML y Ktor Client JS. El bundle de producción pesa 1,19 MiB.

Cliente HTTP tipado (B7.3). Cuatro ficheros en el paquete api/: ApiException (excepción con status, errorCode, userMessage), ApiConfig (autodetección de origen: puerto 8081 → backend en 8080, otro → rutas relativas), ApiClient (HttpClient con ContentNegotiation, método genérico get<T>(): Result<T>, construcción de query strings con filtrado de nulos)

y MovieApi (métodos para cada endpoint con conversión de enums a parámetros camelCase).

Pantalla de listado (B7.4). Sistema de diseño en CSS custom properties: paleta oscura con acento lavanda (#a78bfa), tipografía de sistema, escala de 4px. Componente MovieCard con pósteres de TMDb (CDN w300) y fallback onerror para imágenes expiradas. Grid responsive con repeat(auto-fill, minmax(180px, 1fr)). Estado sealed class (Loading, Success, Error) con LaunchedEffect y reloadKey para retry. Barra de filtros con géneros del endpoint GET /genres. Infinite scroll mediante DisposableEffect con listener de window.scroll, umbral de 300px, flag isLoadingMore anti-duplicados. Botón flotante back-to-top.

Detalle y navegación (B7.5). Navegación hash-based (/#/movie/{id}) en singleton AppState integrado con window.history.pushState y popstate. Pantalla con hero (póster 300px + info), sinopsis, grid de 12 primeros actores, y fila horizontal scrollable de películas similares (de GET /movies/{id}/similar). MovieCard y MovieGrid extendidos con callback onClick opcional.

Búsqueda con autocompletado (B7.6). NavBar persistente (sticky) con campo de búsqueda. LaunchedEffect con debounce de 300ms dispara searchMovies(pageSize=8). Dropdown absoluto con miniatura, título, año, rating y géneros por sugerencia. Enter navega a SearchScreen con 50 resultados.

Tema claro/oscuro (B7.7). Clase CSS .light-theme sobreescribe las custom properties. Estado en AppState.isDarkTheme con persistencia en localStorage y respeto a prefers-color-scheme.

Servir desde Ktor (B7.8). Tarea Gradle copyFrontend copia bundle de producción al classpath del servidor. staticResources("/", "static") en la configuración de rutas. Tarea runDev de tipo JavaExec con carga de .env y modo desarrollo. Cuatro Run Configurations versionadas en .run/.

Problemas encontrados

- Agotamiento de memoria del compilador Kotlin/JS. Al incorporar Ktor Client, el compilador falló con OutOfMemoryError al deserializar las bibliotecas .klib. Se elevó el heap del daemon de Kotlin a 4 GiB en gradle.properties.
- Expiración de pósteres en el CDN de TMDb. Un número significativo de películas no mostraba su póster porque las rutas almacenadas en el dataset habían caducado (TMDb actualiza sus imágenes periódicamente). Se implementó un

fallback en el frontend: cada `<img>` incluye un atributo `onerror` que oculta la imagen rota y revela un elemento hermano con emoji y título. Posteriormente, se actualizaron 42.000 rutas de póster desde un dataset TMDb más reciente.

- Propagación de variables de entorno en Gradle. El plugin Ktor no propagaba las variables de entorno desde IntelliJ al proceso forked. Se creó una tarea `runDev` propia con control explícito del classpath y lectura directa del fichero `.env`.

8. Conclusiones

8.1 Grado de cumplimiento de los objetivos

Objetivo general: diseñar y construir una plataforma web funcional que permita explorar un catálogo de películas y recibir recomendaciones personalizadas basadas en similitud semántica. El objetivo se considera cumplido. El sistema entregado permite al usuario explorar un catálogo de más de cuarenta y cinco mil películas con filtros por género, ordenación y búsqueda textual, consultar el detalle de cada película con su reparto y metadatos, y recibir recomendaciones de películas similares calculadas mediante embeddings semánticos de un modelo Transformer pre-entrenado. La plataforma se ejecuta como un proceso único que sirve tanto la API como la interfaz web desde un solo puerto, facilitando su despliegue y demostración.

- El objetivo de construir un pipeline de datos automatizado y reproducible para la adquisición, limpieza y carga de un dataset real se cumple: los trece scripts Python ejecutan la cadena completa desde los CSV originales hasta la base de datos poblada, incluyendo la generación de embeddings, sin intervención manual más allá de la configuración inicial.
- El objetivo de diseñar un esquema relacional normalizado que modele las entidades del dominio y sus relaciones se cumple: el esquema PostgreSQL sigue la tercera forma normal con tablas de unión para las relaciones muchos-a-muchos, versionado mediante trece migraciones SQL idempotentes gestionadas por un runner propio.
- El objetivo de implementar un sistema de recomendación basado en contenido mediante embeddings semánticos se cumple: el modelo `all-mpnet-base-v2` genera vectores de 768 dimensiones para cuarenta y cuatro mil películas, almacenados en

PostgreSQL con pgvector e indexados con HNSW, con tiempos de consulta de aproximadamente dieciséis milisegundos tras la primera petición.

- El objetivo de exponer los datos y las recomendaciones mediante una API HTTP documentada se cumple: el backend Ktor expone cinco endpoints (listado paginado con filtros, detalle, búsqueda, recomendaciones similares y listado de géneros) con respuestas tipadas, manejo de errores estructurado y una suite de treinta pruebas de integración.
- El objetivo de construir una interfaz web que consuma la API y ofrezca una experiencia de exploración visual se cumple: la aplicación Compose HTML implementa listado con scroll infinito, detalle con recomendaciones, búsqueda con autocompletado en tiempo real y alternancia de tema claro/oscuro.
- Objetivos de aprendizaje. El proyecto ha permitido adquirir experiencia práctica en tecnologías no cubiertas por el currículo del ciclo: Kotlin Multiplatform (compilación a JVM y JavaScript desde el mismo código fuente), programación declarativa de interfaces (Compose), bases de datos vectoriales (pgvector, índices HNSW), modelos de procesamiento del lenguaje natural (sentence-transformers), y gestión de proyectos multi-módulo con Gradle. Estos aprendizajes se documentan a lo largo de la sección 7 con las decisiones técnicas y dificultades que los motivaron.

8.2 Lecciones aprendidas

El cuello de botella del aprendizaje automático basado en contenido es el contenido, no el modelo. La iteración sobre la elección del modelo de embeddings (documentada en la subsección 7.3.4.4) demostró que la calidad de las recomendaciones depende más de la riqueza del texto fuente que de la sofisticación del modelo. Las recomendaciones para películas cuya sinopsis describe adecuadamente sus temas fueron excelentes; las recomendaciones para películas con sinopsis literales o escuetas fueron mediocres independientemente del modelo utilizado. Esta observación tiene implicaciones prácticas directas: antes de invertir en modelos más complejos, conviene invertir en enriquecer los datos de entrada.

Las herramientas generan fricción cuando se combinan de formas no previstas por sus autores. La combinación de Kotlin Multiplatform, Compose para la Web, Ktor y Gradle reveló incompatibilidades no documentadas en cada punto de contacto: accessors deprecados silenciosamente entre versiones menores, tareas Gradle que no propagan variables de entorno al proceso forked, compiladores que agotan la memoria con

dependencias que individualmente son ligeras. La resolución de estos problemas consumió más tiempo del previsto y rara vez produjo conocimiento transferible (la solución a un conflicto de versiones entre Kotlin 2.3.0 y Ktor 3.4.3 no aplica a ninguna otra combinación). La lección es presupuestar tiempo de integración explícitamente al combinar tecnologías de ecosistemas distintos.

Los datos estáticos caducan. El uso de un dataset fijo de TMDB como fuente de datos reveló que las URLs de pósteres almacenadas en el dataset original habían expirado parcialmente: TMDB actualiza y elimina imágenes de su CDN de forma periódica. La solución técnica (un fallback visual en el frontend) fue sencilla, pero la observación tiene implicaciones de diseño: cualquier sistema que almacene referencias a recursos externos debe prever su invalidación. Una ampliación natural del proyecto consistiría en actualizar periódicamente los datos desde la API de TMDB.

8.3 Ampliaciones futuras

Pantalla de persona (actor/director) con filmografía. La base de datos contiene las tablas `people`, `movie_cast` y `movie_crew` con los créditos completos de cada película. Un nuevo endpoint `GET /people/{id}` que devolviera la información de una persona junto con su filmografía, y una pantalla correspondiente en el frontend, permitiría al usuario navegar el catálogo desde la perspectiva del talento creativo además de desde la perspectiva de la película. La arquitectura actual soporta esta ampliación sin modificaciones estructurales: el endpoint seguiría el mismo patrón `Repository/Service/Routes` del backend, y la pantalla seguiría el patrón `screens/components` del frontend.

Filtrado colaborativo. El dataset original incluye un archivo `ratings.csv` con más de veintiséis millones de valoraciones de usuarios de MovieLens, declarado fuera del alcance en la fase de planificación. La incorporación de estos datos permitiría implementar un sistema de recomendación colaborativo (basado en las preferencias de usuarios similares) que complementara al sistema de recomendación basado en contenido ya implementado. Un sistema híbrido que combinara ambas aproximaciones produciría recomendaciones de mayor calidad, especialmente para películas cuya sinopsis es insuficiente para capturar su atractivo (el caso de *Fight Club* documentado en 7.3.4.4).

Actualización periódica del dataset. El uso de un dataset estático introduce dos limitaciones: las películas estrenadas después de la fecha del dataset no aparecen en el catálogo, y las URLs de pósteres pueden expirar. Una ampliación natural consistiría en implementar un proceso de actualización periódica que consultara la API pública de

TMDB (con clave de acceso propia), descargara las películas nuevas y actualizará las existentes. El pipeline de limpieza y carga ya implementado está diseñado para ser idempotente, lo que facilitaría la incorporación de actualizaciones incrementales.

Despliegue en la nube. El sistema actual está diseñado para ejecución local. Un despliegue en un servicio de nube (por ejemplo, un contenedor Docker en un servicio como Fly.io o Railway) requeriría: empaquetar el servidor como un JAR ejecutable (la tarea buildFatJar del plugin Ktor ya lo soporta), configurar una base de datos PostgreSQL gestionada con pgvector (disponible en Supabase, Neon o Amazon RDS), y ajustar las variables de entorno para apuntar a la base de datos remota. La arquitectura de proceso único (backend + frontend estático) simplifica este despliegue al requerir un solo contenedor.

Autenticación y listas personales. La incorporación de un sistema de autenticación (por ejemplo, mediante Firebase Auth como en el proyecto referente) permitiría almacenar listas personales de películas favoritas, un historial de películas consultadas y preferencias de filtrado. Estas funcionalidades transformarían la plataforma de una herramienta de exploración anónima a una herramienta personalizada, lo que a su vez enriquecería el sistema de recomendación con señales de preferencia explícitas del usuario.

9. Bibliografía

Dataset

Banik, R. (2017). The Movies Dataset. Kaggle. Licencia CC0 1.0 Dominio Público.
<https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset>

Documentación oficial de tecnologías

JetBrains. (2026). Kotlin Programming Language — Documentation.
<https://kotlinlang.org/docs/home.html>

JetBrains. (2026). Ktor Framework — Documentation. <https://ktor.io/docs/welcome.html>

JetBrains. (2026). Compose Multiplatform — Documentation.
<https://www.jetbrains.com/compose-multiplatform/>

pgvector contributors. (2026). pgvector: Open-source vector similarity search for PostgreSQL. <https://github.com/pgvector/pgvector>

The PostgreSQL Global Development Group. (2026). PostgreSQL 16 Documentation.
<https://www.postgresql.org/docs/16/>

UKPLab. (2026). sentence-transformers: Multilingual Sentence, Paragraph, and Image Embeddings using BERT & Co. <https://www.sbert.net/>

Hugging Face. (2026). sentence-transformers/all-mpnet-base-v2 — Model Card.
<https://huggingface.co/sentence-transformers/all-mpnet-base-v2>

pandas Development Team. (2026). pandas: Python Data Analysis Library.
<https://pandas.pydata.org/docs/>

Zaxxer. (2026). HikariCP: A solid, high-performance, JDBC connection pool.
<https://github.com/brettwooldridge/HikariCP>

Astral. (2026). uv: An extremely fast Python package installer. <https://docs.astral.sh/uv/>